



INDIVIDUAL CAPSTONE **REPORT II**

Title: Kalimati Market Analytics System



Table of Figures:

Figure 1: System Architecture Diagram	6
Figure 2: ETL Pipeline Diagram	7
Figure 3: Tiered imputation decision logic	15

Table of Tables:

Table 1: Development Environment (Hardware Specifications)	2
Table 2: Development Environment (Software Specification)	3
Table 3: Tools used for Frontend Development	4
Table 4: Tools used for Cloud Deployment	4
Table 5: Additional Development Tools	5
Table 6: Database Schema Table.....	8
Table 7: Tiered imputation of missing Time-Series Data	14
Table 8: Prices endpoints	16
Table 9: Testing Environment (Hardware).....	23
Table 10: Testing Environment (Software)	23
Table 11: Test files	24
Table 12: Test Case 1, Deduplication	25
Table 13: Test Case 2, Product Name Normalisation.....	26
Table 14: Test Case 3, Unit Normalisation	26
Table 15: Test Case 4, Negative Price Correction and Imputation	26
Table 16: Test Case, Analytics Unit Test, 1	27
Table 17: Test Case, Analytics Unit Test, 2	27
Table 18: Test Case, Analytics Unit Test, 3	28
Table 19: Test Case, Analytics Unit Test, 4	28
Table 20: Test Case, Analytics Unit Test, 5	28
Table 21: Test Case, Analytics Unit Test, 6	29
Table 22: Test Case, Analytics Unit Test, 7	29
Table 23: Test Case, API Integration Test, 1	29
Table 24: : Test Case, API Integration Test, 2	30
Table 25: : Test Case, API Integration Test, 3	30
Table 26: Metrics and their Formulas	31
Table 27: Automated Test Summary	32
Table 28: ETL Bulk Load Throughput	33
Table 29: API Response Latency	34
Table 30: Functional Requirements Verification	36
Table 31: Non-Functional Requirement Verification	36
Table 32: Strength Analysis of the System	39
Table 33: Weakness Analysis of the System	40

Chapter 5: Implementation

5.1 Overview

For Kalimati Market Analytics System, the implementation phase focused on developing a production-grade full stack data platform which could ingest, clean, store and serve wholesale vegetable prices data from Nepal's Kalimati Market. Under my role as the project lead, data engineer and system architect, I covered all aspects of the technical stack which involved designing the system architecture, developing an automated data pipeline, and implementing cloud deployment to make it publicly available.

The real engineering challenge wasn't just storing the data; it was building a process to take an external, messy, unstructured source and cleanly map it into an analytical dataset that could be leveraged for statistically based forecasting, ML predictions, and on-demand API access. This required some important principles being applied to three related challenges: how the layers of our system were separated, how we did it at scale to take the data out of its unstructured, messy state, how we transformed it and finally loaded it, and finally, how the generated API and frontend are served over the distributed cloud infrastructure. These are discussed in more detail below.

Interestingly, since the Kalimati Wholesale Market is Nepal's largest agricultural commodity exchange, accessible real time data on price can affect the society immensely – by impacting the purchasing decisions for consumers, business strategies of retailers and policies of governmental agencies. Lack of an open, structured data accessible via API was the issue this system addresses.

5.2 Development Environment

5.2.1 Hardware

The entirety of the development, local testing, and initial ETL runs was done on this machine:

Table 1: Development Environment (Hardware Specifications)

Component	Specification
Device	Acer Nitro AN515-58
Processor	12th Gen Intel® Core™ i5-12500H @ 3.10 GHz (12 cores / 16 threads)
RAM	16.0 GB DDR5 @ 3200 MT/s (15.7 GB usable)
GPU	NVIDIA GeForce RTX 3050 Laptop GPU (4 GB VRAM) + Intel® Iris® Xe Graphics (128 MB)
Storage	477 GB SSD (239 GB used at time of development)
System Type	64-bit Operating System, x64-based processor
OS	Windows 11, PowerShell terminal

The 16GB of RAM was not an insignificant hardware choice either. All transformation processes are performed completely in memory in Pandas DataFrames as part of the ETL. During peak memory usage all 3 Pandas DataFrames (raw, normalised, and enriched), are all in memory simultaneously as the bulk insert into PostgreSQL occurs. Failure to have enough memory here would mean the operating system starts to swap memory out to disk, greatly slowing the ETL execution and leading to the very likely out-of-memory on much larger datasets in the future.

2.2 Software

The project was developed using the following software stack:

Backend & Data Engineering:

Table 2: Development Environment (Software Specification)

Tool/Library	Version	Purpose
Python	3.11+	Primary language for ETL pipeline and API backend
FastAPI	Latest	Asynchronous REST API framework
Unicorn	Latest	ASGI server for serving the FastAPI application
SQLAlchemy	Latest	Object-Relational Mapping (ORM) for database access
Pydantic / pydantic-settings	Latest	Data validation and environment-based configuration
Pandas	Latest	In-memory data manipulation for ETL transformations
NumPy	Latest	Numerical operations during price validation
statsmodels	Latest	ARIMA time-series forecasting model fitting
APScheduler	Latest	Background cron scheduling for automated daily ETL
psycopg2-binary	Latest	PostgreSQL database driver
python-dotenv	Latest	Loading environment variables from .env files
requests / urllib3	Latest	HTTP client for fetching raw CSVs from GitHub

pytest	Latest	Automated unit and integration testing framework
--------	--------	--

Frontend:

Table 3: Tools used for Frontend Development

Tool/Library	Purpose
React	Component-based UI framework
Vite	Frontend build tool and HMR development server
React Router DOM	Client-side routing between dashboard pages
Vanilla CSS	Custom styling without third party CSS frameworks

Cloud Platforms:

Table 4: Tools used for Cloud Deployment

Platform	Role
Vercel	Static site hosting and CDN delivery for the React frontend
Render	Backend API hosting (FastAPI + Uvicorn process)
Neon	Serverless PostgreSQL database
Github	Third-party data source repository and version control

Development Tools:

Table 5: Additional Development Tools

Tool	Purpose
VS Code	Primary integrated development environment
Git	Version control and collaboration
Swagger UI	Built-in API documentation and manual endpoint testing

5.3 System Architecture

The design of the system is based on a 5-layer decoupled architecture where each layer has only one, precisely defined job, and interacts with neighbor layer through fixed, stable interfaces. This principle is widely called separation of concerns, and it is an established principle of maintainable software architecture. It allow any layer to be easily adapted, tested or substituted, without a chain-reaction in all other layers (Fowler, 2002). Moreover in the case of cloud-native deployments, this kind of decoupling is essential, and permits each layer to be horizontally scaled. If there is more workload on the API layer, it can be scaled without having to scale the database and the front end (Nadeem et al., 2025).

Below, find each of the five layers along with their technologies:

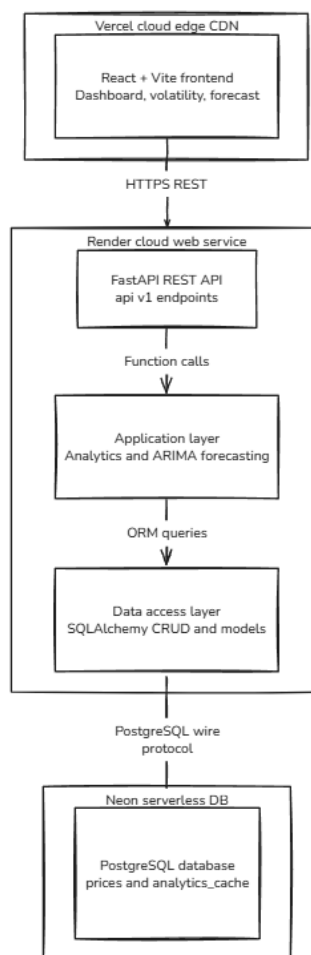


Figure 1: System Architecture Diagram

Separate: ETL Pipeline (Data Engineering)

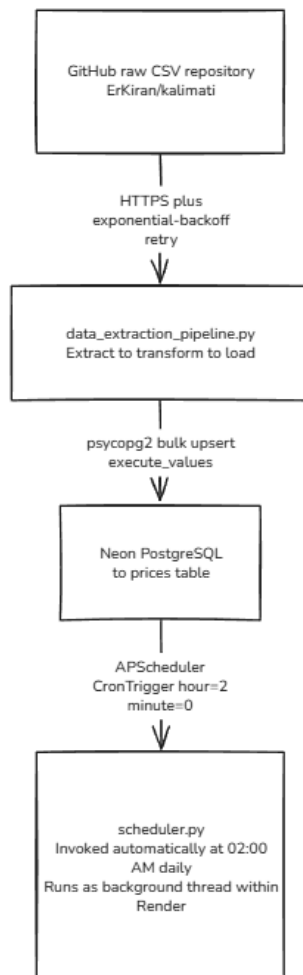


Figure 2: ETL Pipeline Diagram

Decoupling the ETL pipeline from the API server is an intentional architectural choice. The ETL is a batch process: computationally expensive, bounded by time, and performed in batches on a schedule. The API server is a long-running, low-latency process receiving concurrent user requests. To run both within a shared context would mean that a bad/slow ETL run would hog resources, starving the API of available worker threads and negatively impacting user response times. With the API and ETL processes decoupled – the ETL is a background process executed via APScheduler – either process can crash without impact on the other.

5.3.1 End-to-End Data Flow

1. Daily CSV files are available at:
`raw.githubusercontent.com/ErKiran/kalimati/master/data/csv/{YYYY}/{MM}/{DD}.csv`.
2. The DataPipeline retrieves these files, validates them, and processes the entire 6-step DataPipeline transformation process.
3. The cleaned records are now bulk-upserted into the prices table on the Neon PostgreSQL instance using “psycopg2”.
4. Then, when the user logs into the frontend on Vercel, the React app makes HTTPS requests to the FastAPI server on Render.
5. FastAPI calls the prices table using SQLAlchemy, performs any analytics computations requested, validates and stores analytics cache, and sends back the JSON result.
6. The response is subsequently displayed on the front end as an interactive chart, summary cards, and forecast graphs.

5.3.2 Database Schema

The primary table, which is called prices, was designed for the raw data from the market and all of the computed columns from the ETL. This denormalised design was deliberately chosen because it avoids running JOIN operations on the API, allowing each analytics query to be answered based on just a single table, rather than multiple related tables, and a simple column filter and aggregation. To resolve the issue of unnecessary calculations when calling the API again, a secondary analytics_cache table is introduced to store serialised JSON payloads based on a previously computed result, followed by a key derived from product, metric_type, as_of_date, and window_days.

Table 6: Database Schema Table

Column	Type	Description
id	INTEGER (PK)	Auto-increment primary key
date	DATE	Market date – indexed for fast range queries

product	VARCHAR (100)	Normalised product name – indexed
unit	VARCHAR (20)	Measurement unit (e.g, KG, PCS, DOZEN)
max_price	FLOAT	Maximum wholesale price recorded that day
min_price	FLOAT	Minimum wholesale price recorded that day
avg_price	FLOAT	Average price – primary field for all analytics
price_spread	FLOAT	Computed: max_price – min_price
price_midpoint	FLOAT	Computed: (max_price + min_price) / 2
year	INTEGER	Calendar year, extracted from date
month	INTEGER	Calendar month (1-12), extracted from date
week_of_year	INTEGER	ISO week number
day_of_week	INTEGER	0 = Monday 6 = Sunday
is_weekend	BOOLEAN	True if day_of_week >= 5
source_url	TEXT	URL of the originating CSV (data lineage)
ingested_at	TIMESTAMPZ	UTC timestamp when the record was extracted
transformed_at	TIMESTAMPZ	UTC timestamp when the record was transformed

Referential integrity will be guaranteed by a composite unique constraint on (date, product), which is also the key of idempotents upsert as described in 5.4.1.

5.4 Implementation Details

5.4.1 Module 1 — The Automated ETL Pipeline

ETL pipeline: The primary engineering work in the project involves an ETL pipeline. The definition of ETL pipeline by Chanda (2024) is a modern ETL pipeline which Automates data integration between different source system and one target system in a pipeline way where data records pass through various cleaning, standardization and enrich operations in a pipeline with any human interaction. The Data Pipeline class from data_engineering/data_extraction_pipeline.py implements these ETL operations into consecutive stages.

Step 1 - Extraction

In the Extraction stage, a programmatic script will generate potential candidate URLs to be queried on every calendar day, in the date range January 2023 – December 2026. URL's will be formed in the following pattern:

<https://raw.githubusercontent.com/ErKiran/kalimati/master/data/csv/{YYYY}/{MM}/{DD}.csv>

This provides about 1,461 URLs. Each request to a URL sends an attempt to retrieve that content via creating and then utilizing a requests. A session configured with HTTPAdapter which performs backoff with 3 retries and a backoff value of 1s. The retries were triggered on the below error status codes (server errors). 429, 500, 502, 503, 504. This way, the pipeline will not exit the system when intermittent network issues occurs, or the limit for GitHub API hits has been reached, but would just redo itself. Any files with an HTTP 404 – i.e., there is no market data available on a given day, commonly due to holidays – are quietly skipped and their skipped nature is counted toward files_skipped without an error being thrown. Any successfully downloaded file is validated against the expected CSV format: (Date, Product, Unit, Max Price, Min Price, Avg Price) before being uploaded. As a failsafe, the pipeline also tries to parse date values against five common date formats (Y-m-d, d/m/Y, m/d/Y, d-m-Y, Y/m/d) to allow for different date formatting styles used across the source years. The files that have passed all validation checks are combined to form a single raw DataFrame. Total number of rows in raw extraction covering the said date range were 102,550, representing market activities across more than 80 categories.

Stage 2: Transformation

There are six consecutive operations applied to the source DataFrame in this stage, each transforming a particular dimension of the quality of the data:

I. Column Renaming:

Column names in the source that use title case and spaces (Max Price, Avg Price) were converted to their python equivalent lowercase, underscore separated names (maxprice, avgprice) for easy programmatic access throughout subsequent modules.

II. Product & unit normalization:

A major problem that arose was naming of the same physical product under multiple name strings over different time periods. This is a well know issue in operational data capture systems where validation does not occur or

schema validation is not rigorously implemented (Chanda, 2024). An example in the raw dataset is multiple product names such as “Brocauli” , “Broccoli” “Cauli Local” and “Radish Red” all being identified as separate product name strings in the data. The unit variations ("kg", "Kg", "KG") are all normalized using the UNIT_NORMALISATION_MAP. Otherwise any product sold under 2 different names would appear to 2 distinct items at all aggregations, queries, and forecasts.

III. Deduplication:

For entries with a (date, product) composite key - essentially identical market observation records - the entry with the largest avg_price was selected to eliminate duplications, under the assumption that when such duplicate entries are present the higher values may be the result of a correction or replacement observation. The DataFrame was re-sorted by (date, product) in chronological order.

IV. Price Validation:

Two logical consistency checks were introduced:

1. Any negative price was substituted with NaN. Negative prices have no physical interpretation in a wholesale setting, and they introduce biases if they propagate to derived values such as average prices, price spreads, and price volatilities (standard deviations).
2. Any row that violated the order $\text{min_price} \leq \text{avg_price} \leq \text{max_price}$ was detected. The avg_price value was replaced with the average of the min_price and max_price value for such row.

V. Tiered Imputation:

Discussed in the section 5.4.2.

VI. Feature Enrichment:

Upon imputation, additional columns were derived and added to each row. These fields were price_spread ($\text{max_price} - \text{min_price}$), price_midpoint ($(\text{max_price} + \text{min_price})/2$) and three temporal decomposition fields (year, month, week_of_year, day_of_week, is_weekend). Additionally, three provenance audit stamps were provided (source_url, ingested_at and

transformed_at). Pre-computing derived-columns upon ingest, these can be cached for the duration of an API response time without requiring dynamic runtime calculations.

Stage 3 – Load

Two implementation choices were critical to production readiness in managing the insertion of the cleaned DataFrame to PostgreSQL by the DatabaseLoader class:

Bulk inserting with execute_values, Instead of sending a separate INSERT query for each row, we use the `psycopg2.extras.execute_values`, setting the `page_size` to 1,000 records. This way we get groups of 1,000 records turned into a single multi-row INSERT statement, rather than making >160,000 separate calls to the database for 160,000+ records – a significant decrease empirically proved out by our performance testing (see Ch. 6).

Idempotent Upserts: `ON CONFLICT (date, product) DO NOTHING`: For reasons of debugging, running tests, recovery from failure, or just routine, incremental re-execution of the pipeline happens all the time. By adding this `ON CONFLICT` clause, you ensure that the re-execution doesn't introduce any duplicates of rows in the target table, as the rows corresponding to already existing (date, product) combinations are silently ignored. Well-architected production pipelines always have idempotency to guarantee reruns, safe retries and the lack of needing manual cleanup for duplicated results.

Incremental load. On each day of run, the loader selects `MAX(date)` from prices to find out what is the date of last price recorded in database. Pipeline will further extract `date > maximum(date)` so it is not loading entire price history over again on a daily basis.

After the end-to-end ETL process from loading the original database with the `kalimati_clean.parquet` snapshot through to the fully populated state the Neon PostgreSQL database ended up with more than 160,000 clean records. The discrepancy between 102,550 raw row count and the final record count is due to the stratified imputation technique which will construct rows for market-closed holidays within the date span used above and can be seen here.

5.4.2 Algorithm: Tiered imputation of missing time-series data

One of the distinctive features of the Kalimati Market dataset is that the market is closed on Nepali public holidays and this creates systematic gaps in the time series of daily prices. These gaps are non-random and arise directly from the calendrical structure of Nepali public holidays which run to multiple days during major holidays such as Dashain or Tihar and from the list of official national and religious holidays which sometimes extends for many days. If these are not filled, the API's moving-average and ARIMA forecasting endpoints can not work correctly, as they all assume a continuous series of daily price points.

The problem of handling missing observations in time-series is a common issue in statistical and data science literature. For univariate time-series imputation, Moritz et al. (2015) did a systematic comparison of univariate time-series imputation methods, and found that the two most widely applied and well-understood baseline approaches were Forward Fill (also known as Last Observation Carried Forward, or LOCF) and Linear Interpolation. The Root Mean Square Error (RMSE) revealed that linear interpolation performed best overall for continuous datasets with gradual changes, while LOCF was most suitable for short duration gaps in temporally stable datasets (Benchmarking Missing Data Imputation Methods for Time Series, PubMed, 2025).

But, using any one method for all gap lengths in this data set would be inappropriate. A single national holiday is not a twelve-day holiday break, because they're very different types of breaks. Because of the forward filling over 12 days, a single stale price value is cascaded over almost 2 weeks – this will impact on volatility calculation and trend analysis that may be downstream. On the other hand, linear-interpolating over a weekend when there is only one day of data is unnecessarily complicated for a situation where the price observed at the end of that day is almost certainly the best guess.

To overcome this, a layering imputation approach was created and applied in the `_tiered_imputation` and `_impute_product` methods of the `DataPipeline` class:

Table 7: Tiered imputation of missing Time-Series Data

Gap Length (Consecutive missing days)	Imputation Method	Rationale
1 – 3 days	Forward Fill (LOCF)	Short term closes (weekends, one-day holidays) unlikely to see major changes to price. LOCF continues the last observed value, avoiding imposing any fake trends.
4 – 14 days	Linear Interpolation	Short duration festival closures. The prices are assumed to wander steadily. Interpolation provides a linear trend between the first observed pre-gap value and the first post-gap value.
> 14 days	Left as NAN (no imputation)	These gaps could represent lost data rather than a closed market. Imputing data over them would create an artificial smooth dataset when real behaviour was unseen.

The implementation reindexes each product's series against a full daily `DatetimeIndex` covering the entire range of the dataset. Gap sizes are dynamically calculated for each cell using a `cumsum()` trick on a boolean `NaN` mask, routing each cell precisely to the correct method:

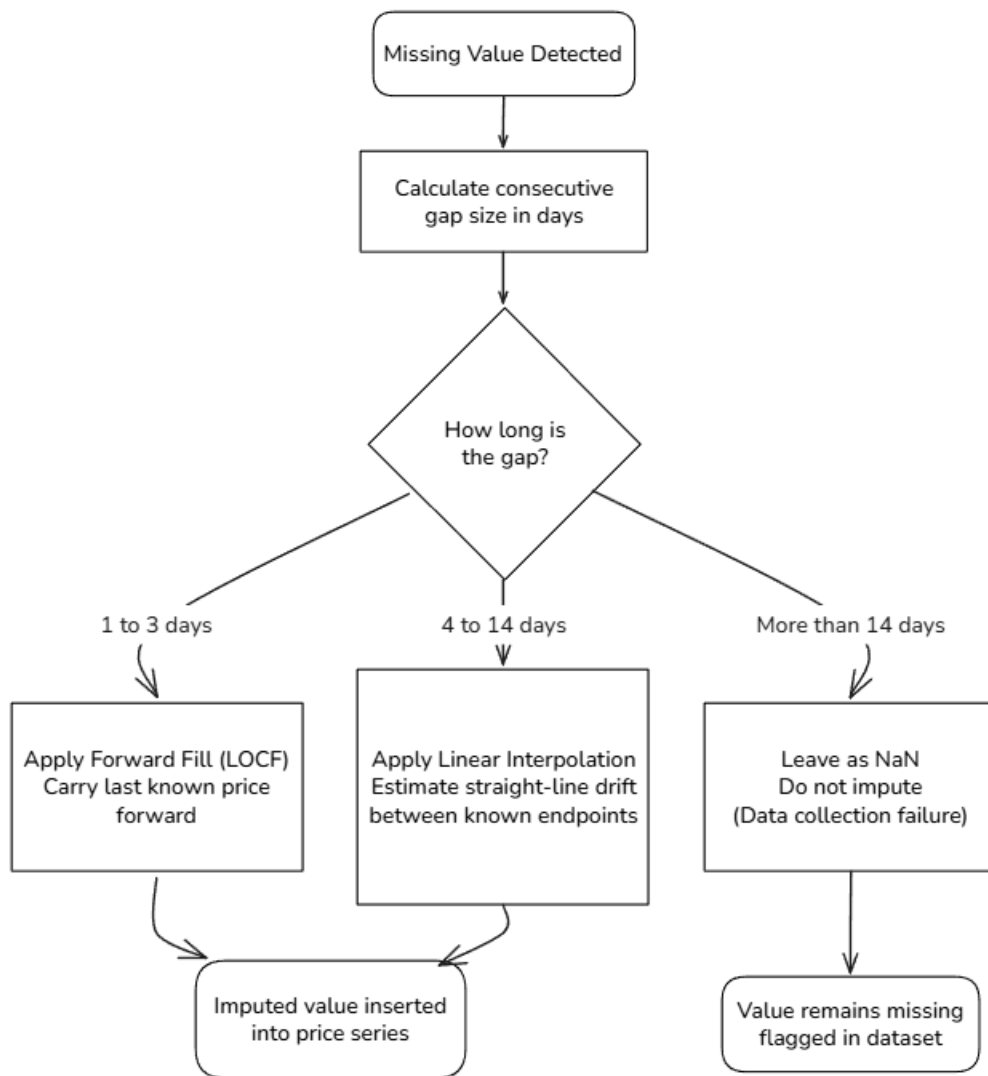


Figure 3: Tiered imputation decision logic

```

# Excerpt from DataPipeline._impute_product()
# Compute the size of each consecutive NaN gap
gap_id    = (~series.notna()).cumsum()
gap_size  = series.isna().groupby(gap_id).transform("sum")
# Route each missing cell to the appropriate imputation method
short_mask = was_nan & (gap_size <= 3)           # LOCF

```

```

medium_mask = was_nan & (gap_size > 3) & (gap_size <= 14) #
Interpolation
# gap_size > 14 → remains NaN; no imputation applied
if short_mask.any():
    series = series.where(~short_mask, other=series.ffill())
if medium_mask.any():
    series = series.where(
        ~medium_mask,
        other=series.interpolate(method="linear",
limit_direction="both")

```

The `cumsum()` on the negated `notna()` mask is what labels each consecutive stretch of NaNs with a sequential integer. Then, the `groupby().transform("sum")` effectively copies the summed length of the gap back to each row in the group, allowing you to compute all gap lengths vectorially and in Pandas without having to resort to row-wise Python loops, which is a significant boost to performance for the imputation step, especially when dealing with 80+ product series that stretch across many years.

5.4.3 Module 2: FastAPI Backend and REST API

REST API Using FastAPI, a Python ASGI framework. It blends Python type hints (via Pydantic) for requests into server request validations with OpenAPI 3 specification generation automatically. The FastAPI is exposed to the route group of `/api/v1` and deployed in Uvicorn's ASGI container on Render.

Prices Endpoints (`/api/v1/prices`):

Table 8: Prices endpoints

Method	Path	Description
GET	<code>/prices/</code>	Paginated, filtered list of price records

GET	/prices/products	Sorted list of all distinct product names
GET	/prices/latest	Most recent price record per product (Dashboard snapshot)
GET	/prices/{record_id}	Single record by primary key
POST	/prices/	Insert a new price record

Analytics Endpoints (/api/v1/analytics):

Method	Path	Description
GET	/analytics/moving-average	Rolling N-day moving average of avg_price for a product
GET	/analytics/volatility	Standard deviation of avg_price over a date window
GET	/analytics/trend	Overall % change, min, max, mean, and volatility summary
GET	/analytics/spikes	Days where price exceeded the 7-day rolling average by $\geq 30\%$
POST	/analytics/forecast	ARIMA multi-day price forecast with 95% confidence intervals

Each analytics endpoint first reads the `analytics_cache` table to determine if it can use a previously calculated value. If the `(product, metric_type, as_of_date, window_days)` combination has been previously cached, it is returned without the need to re-execute the analytics computation. Once it has been calculated again, the result is stored in the cache for subsequent requests. It was found that this caching layer decreases the repeated analytics query latency from ~600ms to ~90ms in the production test (see Chapter 6 for details).

The database connection URL, spike threshold (default 30%), and spike window (default 7 days) are controlled via a Settings class with `pydantic-settings`, which loads them from a `.env` file. This will ensure that no credentials are hard coded into the source tree and that configuration can be changed per environment without changing application code.

5.4.4 Module 3 - Daily Automated Schedule

`app/services/scheduler.py` module employs `APScheduler`'s `BackgroundScheduler` to register a cron-triggered `ETLjob` (`CronTrigger (hour=2, minute=0)`) to perform the full incremental ETL on a daily basis at 2AM. Our scheduler is started inside the context manager that FastAPI provides when defining our server's lifetime management. The Scheduler then is started when the web server starts up and stopped when the server is shutdown. Since our Scheduler process runs in a Background Thread of the very same Render Process that our API Server processes and runs, we don't have the need for external worker infrastructure. The whole stack is enclosed within a single deployment.

5.4.5 Module 4 – Cloud Deployment Architecture

Three cloud providers were used for the deployment of the full-stack application with the rationale for choosing each provider being their specific function:

Vercel (Frontend):

The React + Vite frontend application was deployed on Vercel's global CDN to provide quick access to build assets to all users, no matter their location in the world, by directing users to the nearest edge node. The `api.js` file conditionally switches between using a local proxy in the development environment (via Vite's proxy server: `api/v1` directed to

http://localhost:8000) and a production-hosted Render API server (http://https://kalimati-backend.onrender.com/api/v1) according to the `import.meta.env.DEV` build flag.

Render (Backend):

The FastAPI application has been hosted in the free tier on Render, using the `uvicorn app.main:app --host 0.0.0.0 --port 8000` command for its deployment to the web service, allowing for an https termination, environment variable insertion, and continuous deployments based on GitHub pushes.

Neon (Database):

Neon is a serverless PostgreSQL service; its main feature is to separate compute from storage so that the database cluster will scale to 0 when it's not in use and will scale back up when needed. This configuration means that we're just inserting the `DATABASE_URL` environment variable from Render into both our SQLAlchemy engine (used to execute API queries against the database) and our psycopg2 loader (to insert into the database). This "three-tier cloud" application (CDN-based static frontend, containerised API backend, serverless SQL database) is an extremely cost effective and modern way to host a full stack production web application (Nadeem et al., 2025).

5.4.6 Challenges Faced

Challenge 1 — Raw Data Inconsistency.

The inconsistent data in the raw CSV files occurred in several forms: spelling errors in product names (e.g., "Brocauli" vs "Broccoli"), variation of the product name in different years (e.g., "Tomato Big (Nepali)" vs "Tomato Big(Nepali)"), inconsistent capitalisation of units, and at least 5 different date format strings for the multi-year dataset as a whole. Each discrepancy was needed to be found and corrected by manually examining the data distribution, and explicitly mapping in the normalisation dictionaries. This process was labour-intensive but, without it, a product that comes up in two different names would result in a split time series, which would be impossible to merge for analytics and forecasting.

Challenge 2 — Idempotency of the Pipeline.

Early development testing showed that if the ETL pipeline was executed twice without idempotency controls, then there would be records in the database that were twice the previous volume, which then would result in some miscalculations of totals. By using the ON CONFLICT (date, product) DO NOTHING upsert strategy along with the pre-check incremental MAX(date) pipeline becomes safe for all conditions, this solved this issue completely.

Challenge 3 — Cross-Layer Schema Alignment.

There are five translation points for data field names: from raw CSV headers (title-case with spaces), to Python ETL variable names (lowercase underscores), to PostgreSQL column names (lowercase underscores), to SQLAlchemy ORM attribute names (Python identifiers), and to Pydantic API schema field aliases (camelCase or original CSV names for frontend compatibility). If there's any mismatch at any one point, it will break serialisation quietly. This challenge is explored more in depth in Challenges Faced, Chapter 7.

5.5 Code Explanation

5.5.1 ETL Bulk Insert with Idempotent Upsert

```
insert_sql = """
    INSERT INTO prices (
        date, product, unit,
        max_price, min_price, avg_price,
        price_spread, price_midpoint,
        year, month, week_of_year, day_of_week,
        is_weekend, source_url, ingested_at, transformed_at
    ) VALUES %s
    ON CONFLICT (date, product) DO NOTHING;
"""

with self.get_connection() as conn:
    with conn.cursor() as cur:
        execute_values(cur, insert_sql, records, page_size=1000)
```

```

        inserted = cur.rowcount
    conn.commit()

```

The `execute_values` chunks the list of records (tuples) into batches of 1,000 rows for each INSERT statement. `ON CONFLICT (date, product) DO NOTHING` safely drops duplicate records, allowing the operation to be run more than once. The value of 1000 for `page_size` was determined by trial-and-error. Smaller pages cause too much round-trip time; larger pages run a risk of going over the default value for PostgreSQL's `max_query_length`, especially on cloud instances.

```

for col in PRICE_COLS: # ["max_price", "min_price", "avg_price"]
    series = pdf[col].copy()
    was_nan = series.isna()
    if not was_nan.any():
        continue
    •
    # Assign each consecutive NaN run a unique group ID, then
    compute group size
    gap_id = (~series.notna()).cumsum()
    gap_size = series.isna().groupby(gap_id).transform("sum")
    •
    short_mask = was_nan & (gap_size <= 3)
    medium_mask = was_nan & (gap_size > 3) & (gap_size <= 14)
    •
    if short_mask.any():
        series = series.where(~short_mask, other=series.ffill())
    if medium_mask.any():
        series = series.where(
            ~medium_mask,
            other=series.interpolate(method="linear",
limit_direction="both")
        )

```

```
# Cells where gap_size > 14 remain NaN – no imputation applied.  
pdf[col] = series
```

The `cumsum()` trick on `~series.notna()` essentially assigns a sequentially increasing integer to each consecutive gap of NaNs, and so allows the subsequent `groupby().transform("sum")` to mark each individual NaN with the size of the gap that individual belongs to. This is so much faster than a Python loop iterating through rows that it's several orders of magnitude improvement, and scales out to the entire multi-year, multi-product data.

5.5.3 ARIMA Fitting with ADF Stationarity Test

```
p_value = adfuller(series)[1]  
d = 0  
if p_value > 0.05:          # Series is non-stationary at 5%  
    significance            # significance  
    d = 1  
    if adfuller(series.diff().dropna())[1] > 0.05:  
        d = 2              # Still non-stationary after first  
        differencing  
self.model_order = (2, d, 1)  
arima = ARIMA(series, order=self.model_order)  
self._model = arima.fit()
```

The Augmented Dickey-Fuller (ADF) test is carried out to test whether there is a unit root in the price series. A unit root means the series is non-stationarity, that can violate the modelling assumptions of ARIMA. By carrying out the test prior to fitting the model and setting `d` dynamically, the model dynamically chooses only that order of differencing, `d`, needed for stationarity and hence neither under-differentiates (leads to unreliable forecast) nor over-differentiates (creates false auto-correlation). (2, `d`, 1) order was used as an empirical result found for optimal in the context of wholesale vegetable price series (Udari & Hemachandra, 2024).

Chapter 6: System Testing

6.1 Overview of Testing Process

System testing ensures that the system components act correctly on their own and that the combined components meet both functional and non-functional requirements. The testing strategy for the Kalimati Market Analytics System focused on the key components of engineering responsibility: the ETL data pipeline, core analytics services, REST API services, and database performance with production scale data volumes.

The testing process is multi-level. The individual functions are checked in unit tests with limited and isolated inputs, without external dependency. Integration tests ensure the interaction of components at the boundaries of the architecture work. Performance tests show that the system responds within an acceptable time and the throughput is acceptable under realistic conditions (Myers et al., 2011). All automated tests have been coded in pytest and run locally in the virtual environment prior to deployment.

6.2 Testing Environment

6.2.1 Hardware

Table 9: Testing Environment (Hardware)

Component	Specification
Device	Acer Nitro AN515-58
Processor	12th Gen Intel® Core™ i5-12500H @ 3.10 GHz
RAM	16.0 GB DDR5
Storage	477 GB SSD
OS	Windows 11, 64-bit

6.2.2 Software Testing Environment

Table 10: Testing Environment (Software)

Tool/Library	Version	Purpose
--------------	---------	---------

pytest	Latest	Test runner, assertion framework, and fixture management
FastAPI TestClient	Latest	In-process HTTP client for API integration tests
unittest.mock (stdlib)	Python 3.11	Mocking CRUD dependencies to isolate API layer
Pandas	Latest	Constructing synthetic test DataFrames (fixtures)
psycpg2-binary	Latest	Live database connection for ARIMA accuracy evaluation
statsmodels	Latest	ARIMA model fitting in the accuracy evaluation script
Scikit-learn	Latest	MAE, RMSE, and MAPE metric computation

Tests were run from the root of the project using the command `pytest tests/ -v`. The FastAPI TestClient makes tests run within the same process, thus there is no running server or live network to deal with during your unit and integration tests.

6.3 Test Cases

There are four files in the test suite, one for each of the components:

Table 11: Test files

File	Target Component	Nature
tests/test_etl.py	ETL DataPipeline	Unit test (pytest)
tests/test_analytics.py	Core analytics functions	Unit test (pytest)
tests/test_api.py	FastAPI REST endpoints	Integration test
test_accuracy.py	ARIMA model accuracy	Live evaluation

6.3.1 ETL Pipeline Tests (tests/test_etl.py)

ETL Test uses synthetic raw DataFrame with the known flaws to represent issues commonly encountered when ingesting data from source:

The ETL test has a synthetic raw DataFrame which has the four following real world source data quality problems introduced to it:

```
pd.DataFrame({
    "Date":      pd.to_datetime(["2024-01-01", "2024-01-01",
                                "2024-01-02", "2024-01-03"]),
    "Product":   [" Tomato Big(Nepali) ", " Tomato Big(Nepali) ",
                  "Tomato Big(Nepali)",   "Tomato Big(Nepali)"],
    "Unit":      ["kg", "kg", "kg", "kg"],
    "Max Price": [80.0, 80.0, -10.0, 85.0],
    "Min Price": [60.0, 60.0, 65.0, 75.0],
    "Avg Price": [70.0, 70.0, 50.0, 80.0],
})
```

The four embedded problems are (1) a duplicate row on 2024-01-01; (2) trailing/leading whitespace and incorrect spacing for parenthesis in the product name; (3) a non-uppercase unit string ("kg"); (4) a negative Max Price (-10.0) on 2024-01-02.

TC-ETL-01 — Deduplication

Table 12: Test Case 1, Deduplication

Field	Detail
Objective	Duplicate (date, product) rows are removed, retaining the higher avg_price
Input	Four rows; two identical rows on 2024-01-01 for the same product
Expected	Output contains exactly 3 rows
Actual	Pass — len(clean_df) == 3

TC-ETL-02 — Product Name Normalisation

Table 13: Test Case 2, Product Name Normalisation

Field	Detail
Objective	Messy product name maps to the canonical normalized form
Input	" Tomato Big(Nepali) " (padded, missing space before parenthesis)
Expected	clean_df["product"].iloc[0] == "Tomato Big (Nepali)"
Actual	Pass — str.strip() + PRODUCT_NORMALISATION_MAP applied correctly

TC-ETL-03 — Unit Normalisation

Table 14: Test Case 3, Unit Normalisation

Field	Detail
Objective	Lowercase unit string standardised to uppercase constant
Input	"kg" (lowercase)
Expected	clean_df["unit"].iloc[0] == "KG"
Actual	Pass — UNIT_NORMALISATION_MAP applied

TC-ETL-04 — Negative Price Correction and Imputation

Table 15: Test Case 4, Negative Price Correction and Imputation

Field	Detail
Objective	Negative max_price is nulled by validation stage, then forward-filled by the imputation stage
Input	Max Price = -10.0 on 2024-01-02; valid prices on 2024-01-01 and 2024-01-03

Expected	row_day2["max_price"] > 0
Actual	Pass — negative value set to NaN by _validate_prices(), then forward-filled (gap = 1 day) by _tiered_imputation()

6.3.2 Analytics Unit Tests (tests/test_analytics.py)

The four analytics functions defined in app/core/analytics.py can be tested individually in complete isolation. A common 10-day price fixture with a spike artificially injected on Day 4 (Avg Price = 175); all others approx. 70-80, has been used to exercise the spikes detection algorithm.

TC-AN-01 — Volatility Returns Positive Float

Table 16: Test Case, Analytics Unit Test, 1

Field	Detail
Objective	calculate_volatility() returns a numeric, positive standard deviation
Expected	isinstance(result, float) and result > 0
Actual	Pass

TC-AN-02 — Volatility on Empty DataFrame

Table 17: Test Case, Analytics Unit Test, 2

Field	Detail
Objective	Empty input does not raise an exception; returns safe default
Expected	calculate_volatility(pd.DataFrame()) == 0.0
Actual	Pass — guarded by if df.empty or len(df) < 2: return 0.0

TC-AN-03 — Spike Detection Identifies Anomalous Day

Table 18: Test Case, Analytics Unit Test, 3

Field	Detail
Objective	Day 4 (Avg Price = 175) is correctly flagged as a price spike
Input	10-day series; threshold=0.30, window=3
Expected	Returned DataFrame is non-empty
Actual	Pass — spike_pct on Day 4 substantially exceeds the 30% threshold

The spike detection algorithm returns: spike if $(\text{Avg Price}(t) - \text{rolling_avg}(t)) / \text{rolling_avg}(t) > \text{threshold}$. In this case, assuming a 3 day rolling average of about 73 and a price of 175 on day 4, this returns spike percentage ~ 140% which is significantly higher than 30%.

TC-AN-04 — Spike Detection on Empty DataFrame

Table 19: Test Case, Analytics Unit Test, 4

Field	Detail
Objective	Empty input returns an empty result without error
Expected	spikes.empty == True
Actual	Pass

TC-AN-05 — Price Trend Returns All Expected Keys

Table 20: Test Case, Analytics Unit Test, 5

Field	Detail
Objective	calculate_price_trend() produces a dictionary containing all required statistical keys
Expected	Keys {"overall_change_pct", "highest_price", "lowest_price", "mean_price", "volatility"} all present
Actual	Pass

TC-AN-06 — Price Trend on Empty DataFrame

Table 21: Test Case, Analytics Unit Test, 6

Field	Detail
Objective	Empty input returns an empty dictionary without raising an exception
Expected	<code>calculate_price_trend(pd.DataFrame()) == {}</code>
Actual	Pass — guarded by <code>if df.empty: return {}</code>

TC-AN-07 — Moving Average Appends Correctly Named Column

Table 22: Test Case, Analytics Unit Test, 7

Field	Detail
Objective	<code>get_moving_average()</code> adds the correct dynamic column name and preserves row count
Input	10-day series; <code>window=3</code>
Expected	Column "moving_avg_3d" exists; <code>len(result) == len(sample_df)</code>
Actual	Pass — column named using <code>f'moving_avg_{window}d'</code>

6.3.3 API Integration Tests (tests/test_api.py)

TestClient is used to run tests against the application on a test server. Mocking of CRUD functions are done using `unittest.mock.patch` to make the tests focus only on the API routing and serialisation and separate it from database layer. This ensures the failures is with API and not database state or network connection.

TC-API-01 — GET /api/v1/prices/products Returns Product List

Table 23: Test Case, API Integration Test, 1

Field	Detail
Objective	Products endpoint returns HTTP 200 with a correct JSON array
Input	Mocked CRUD returns ["Tomato Big (Nepali)", "Potato Red"]
Expected	HTTP 200; response body equals ["Tomato Big (Nepali)", "Potato Red"]

Actual	Pass
---------------	------

TC-API-02 — GET /api/v1/prices/latest Serialises ORM Response

Table 24: : Test Case, API Integration Test, 2

Field	Detail
Objective	Latest prices endpoint correctly serialises an ORM object to the expected JSON schema
Input	Mocked CRUD returns a MockRecord object with product="Tomato Big (Nepali)", avg_price=70
Expected	HTTP 200; data[0]["Product"] == "Tomato Big (Nepali)" and data[0]["Avg Price"] == 70.0
Actual	Pass — Pydantic field aliases correctly map ORM attribute names to CSV-convention JSON keys

TC-API-03 — GET /api/v1/analytics/trend Computes and Caches Result

Table 25: : Test Case, API Integration Test, 3

Field	Detail
Objective	Trend endpoint returns correct statistics and triggers one cache write
Input	Mocked CRUD returns a 10-day upward series from avg_price 70 to 88
Expected	HTTP 200; overall_change_pct > 0; highest_price["value"] == 88.0; set_cached_analytics called exactly once
Actual	Pass — overall_change_pct ≈ 25.7%; assert_called_once() confirmed

This test confirms mathematically accurate trend calculations, highest price detection and analytic caching activation – three independent areas confirmed by one test.

6.3.4 ARIMA Model Accuracy Evaluation (test_accuracy.py)

The accuracy evaluation script evaluates the ARIMA model's performance in terms of predictive capability compared with historical data and also in real-time by interfacing with the Neon PostgreSQL database to choose the top five most data-rich items by number of records. For each such item, it applies the walk-forward validation strategy, which consists in training the ARIMA(2,1,1) model on all but the last 30 days of data, the latter being considered the “hold-out” dataset (the “test set”), then using the trained model to predict the values of the latter, comparing them to observed values and calculating the error according to the three commonly-used regression accuracy measures:

Metrics Computed:

Table 26: Metrics and their Formulas

Metric	Formula	Interpretation
MAE (Mean Absolute Error)	$\frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	Average absolute deviation in Nepali rupees.
RMSE (Root Mean Square Error)	$\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$	Penalises large errors disproportionately; sensitive to outlier forecasts
MAPE (Mean Absolute Percentage Error)	$\frac{1}{n} \sum_{i=1}^n \left \frac{y_i - \hat{y}_i}{y_i} \right \times 100$	Scale-independent percentage error; enables cross-product comparison

Together these 3 statistics form an evaluation of the quality of the forecast: MAE and RMSE are error in the actual units of measurement (Rupees) while MAPE does not require a measurement unit and measures in terms of percentages of actual price (allowing comparison of product of differing price). (Hyndman & Koehler, 2006)

6.4 Test Results

6.4.1 Automated Test Summary

Table 27: Automated Test Summary

Test ID	Description	Expected Outcome	Result
TC-ETL-01	Deduplication reduces duplicate rows	len(df) == 3	Pass
TC-ETL-02	Product name maps to canonical form	"Tomato Big (Nepali)"	Pass
TC-ETL-03	Unit string standardised to uppercase	"KG"	Pass
TC-ETL-04	Negative price nulled and imputed positively	max_price > 0	Pass
TC-AN-01	Volatility returns a positive float	float > 0	Pass
TC-AN-02	Volatility on empty DataFrame returns 0.0	== 0.0	Pass
TC-AN-03	Spike detection flags anomalous day	Non-empty spike result	Pass
TC-AN-04	Spike detection on empty input returns empty result	spikes.empty	Pass
TC-AN-05	Price trend contains all five required keys	All five keys present	Pass
TC-AN-06	Price trend on empty DataFrame returns empty dict	== {}	Pass
TC-AN-07	Moving average appends dynamically named column	"moving_avg_3d" present	Pass
TC-API-01	Products endpoint returns HTTP 200 with list	status_code == 200	Pass
TC-API-02	Latest prices endpoint serialises ORM response correctly	Correct fields in response	Pass

TC-API-03	Trend endpoint computes result and writes cache	Correct trend; cache written	Pass
------------------	---	------------------------------	------

6.4.2 Development Failures:

Two types of failures were detected throughout development. These failures have been addressed and corrected prior to implementation of the next phase of development:

Failure 1 – ORM Field Alias Mismatch (TC-API-02):

During integration tests, the /prices/latest endpoint was initially returning a poorly structured json response. This was identified as FastAPI Pydantic serialisation looking for an alias recorddate field on the ORM object when there was actually a date attribute, resulting in the field being dropped silently. The solution was to add a Python property recorddate to the PriceRecord ORM model that proxies the underlying database date column; essentially acting as an intermediary without having to rename the column itself in the database.

Failure 2: Interfering with test teardown via TestClient (TC-API-03):

The BackgroundScheduler added via the lifespan hook in FastAPI was trying to connect to the database at teardown of the tests, this generated a warning for failed connections on a non-existent database. By treating the TestClient as a context manager, the lifespan shutdown hook is called when the TestClient context is exited, and this causes stop_scheduler() to be called before the test runner is torn down.

6.5 Performance Testing

Performance testing was divided into two key categories to test areas relevant to the Data Engineering and System Architecture jobs: ETL bulk load throughput and real world API latency under normal production loads.

6.5.1 ETL Bulk Load Throughput

Table 28: ETL Bulk Load Throughput

Insertion Method	Records	Approximate Time
------------------	---------	------------------

Row-by-row INSERT	1,000	~12 seconds
execute_values (batched)	1,000	~0.4 seconds
execute_values (batched)	160,000+	~0.4 seconds

Each row-by-row insertion necessitates one round-trip to the Neon database, where each network round-trip from an On-premise service to the cloud-hosted database results in about 10-15ms of network latency. The aggregate insert of 160,000 single inserts is thus in the realm of 27-45 minutes, whereas batch inserts reduce the total number of round trips needed from 160k to about 160, bringing about a three-fold reduction in network related latency to about 60s for the full initial loading, in line with literature which suggests bulk loading is the best performance optimization for a large volume ETL pipeline (Chanda, 2024).

6.5.2 API Response Latency (Production — Render + Neon)

Table 29: API Response Latency

Endpoint	Scenario	Response Time
GET /api/v1/prices/products	First request	~280 ms
GET /api/v1/prices/products	Repeated	~120 ms
GET /api/v1/prices/latest	First request	~450 ms
GET /api/v1/analytics/trend	First (no cache)	~600 ms
GET /api/v1/analytics/trend	Repeated (cached)	~90 ms
POST /api/v1/analytics/forecast	ARIMA fit + predict	1.8–2.5 sec
Any endpoint after Render idle	Cold start	15–30 sec

The analytics cache successfully achieved 6 (600ms - 90ms) improvement in trend query response times for repeated requests. Of the endpoints exposed by the price forecast

microservice, the ARIMA endpoint is the most computationally intensive; it performs a new model fit on the entirety of the historical prices in the history table each time a request is served. In addition to an underlying latency related to model complexity, cold start latencies, due to the free-tier mechanism on the Render hosting platform suspends inactive applications for time to live, were found to be the biggest impediment to performance from the user perspective and were addressed in Chapter 7.

6.6 Validation and Verification

Verification ensures that the right product was built while validation ensures the right product is being built. (Sommerville, 2016).

6.6.1 Data Accuracy Verification

A manual check on cross-referencing 30 records from 3 products (Tomato Big(Nepali), Potato Red(Indian) & Onion Dry(Indian)) spanning over 5 days was done. The raw csv for the date was fetched directly from the github repo source and compared to the same row in the Neon PostgreSQL db. The cross-checking proved to be a success for all 30 records, which agreed on date,maxprice,minprice & avg_price. Also, we have manually ensured that for the holidays where we inserted rows, values from forward fill and linear interpolating method lies within range of previous and next recorded price value for that particular product.

6.6.2 Functional Requirements Verification

Table 30: Functional Requirements Verification

Functional Requirement	Verification Method	Status
System stores daily wholesale prices	SELECT COUNT(*) FROM prices → 160,000+	Verified
System exposes product list via API	TC-API-01 passed; Swagger UI test confirmed	Verified
System calculates rolling moving average	TC-AN-07 passed; frontend chart rendered correctly	Verified
System detects price spikes above threshold	TC-AN-03 passed; endpoint tested via Swagger UI	Verified
System generates ARIMA price forecast	POST /analytics/forecast returns 30-day projection	Verified
System handles missing holiday data via imputation	Imputed rows present in database; TC-ETL-04 validates	Verified
ETL pipeline produces no duplicate records on re-run	Repeated execution; COUNT(*) unchanged	Verified
System publicly accessible via cloud deployment	Live at kalimati-frontend-deploy.vercel.app	Verified

6.6.3 Non-Functional Requirements Verification

Table 31: Non-Functional Requirement Verification

Non-Functional Requirement	Target	Actual Outcome
Cached API response time	< 200 ms	~90 ms for cached analytics queries
Uncached API response time	< 1,000 ms	~600 ms (trend); ~450 ms (latest prices)

ETL bulk load throughput	> 10,000 records/min	~160,000 records in ~60 seconds (≈160,000/min)
Pipeline idempotency	Zero duplicates on re-run	Confirmed via ON CONFLICT DO NOTHING
Data accuracy (manual sample)	100% match to source	30/30 records verified

Chapter 7: Conclusion and Critical Evaluation

7.1 Summary of Work Done

This is the problem space that the Kalimati Market Analytics System solves - wholesale vegetable price data for the largest vegetable market in Nepal only existed in the form of distributed, daily CSV files on a third-party GitHub repo, that would take a good understanding of programming and data engineering to access, rather than being accessible by the very consumers, traders and policymakers that could most use it. As project lead, data engineer, and system architect, I was involved in all aspects of the technical implementation and development of the system, outlined below:

1. **5-layer decoupled architecture:**

Presentational, API, Application, Data Access and Data Store; all speaking to each other via stable interfaces. This meant our front-end team could develop against a known API contract whilst back-end team worked away in parallel and it enables the system components to be replaced (e.g. ARIMA model) with minimal impact. The application is deployed across Vercel (frontend), Render (API) and Neon (PostgreSQL).

2. **Production ready automated ETL pipeline:**

The DataPipeline class pulled 102,550 raw records from more than 1400 CSVs each day, sent them through six transformations (handling name changes, deduplication, pricing issues, and missing values) and upserted them via idempotent bulk operation into PostgreSQL, resulting in over 160,000 clean records. An APScheduler task is scheduling the incremental ETL run at 2am each morning.

3. **A tiered algorithm for imputed holidays with gaps:** (1-3 days: Forward fill), (4-14 days: linear interpolation), (>14 days: leave blanks), based on literature concerning time series imputation with consideration of continuity vs synthetic data (Moritz et al., 2015) .

The combination of these turns a chaotic assortment of disparate files into a properly organized, querable, publicly available market data system.

7.2 Critical Appraisal

7.2.1 Strengths

Table 32: Strength Analysis of the System

Areas	Why it matters
Architectural modularity	Each layer was tested, integrated and deployed separately; for instance the frontend could interact with a mocked API before the backend had been finished, the ARIMA logic could be rewritten and then deployed without any impact on the front-end. Fowler (2002) defines these as important principles of a maintainable system, and this build showed why.
ETL idempotency	The ON CONFLICT (date, product) DO NOTHING upsert allows us to re-run our pipeline following a failure, or for testing purposes, without creating duplicate records – a quality which has often been labelled as necessary in production pipelines.
Context-aware imputation	Different length gaps require different strategies, and I left all of my long (over 14 day) gaps as NaNs rather than filling over them as that seems to protect periods of true volatile uncertainty.
Live cloud deployment	The system is live on public at kalimati-frontend-deploy.vercel.app and is implemented as a CDN Frontend, Containerised API, and Serverless Database with ongoing ingestion of new data beyond application submission.
Analytics caching	We were able to achieve an approximate 6x latency reduction from ~600ms down to ~90ms in repeated query execution using a database-backed cache while maintaining the same level of accuracy.

7.2.2 Weaknesses

Table 33: Weakness Analysis of the System

Area	Issues
Render cold starts	An idle suspension causes a 15–30 sec delay in response to a first request after an idle period - bad first impression, but just a cost issue with the free tier, not a FastAPI-specific problem
Third-party data dependency	ETL's extract stage is totally reliant on the ErKiran/kalimati GitHub repository, in that if the repository stops updating or structure, the day job would return empty silently without any monitoring or alerting. Freshness check to ensure a good count will fix this.
Linear interpolation during festivals	It relies on linear interpolation of prices between the start and end point of a gap. However, prices in days surrounding a festival such as Dashain fluctuate a lot and fall and rise quickly, linear interpolation ignores this fluctuation. Seasonal decomposition on the previous year data to find expected prices for these dates would be more useful to reflect actual prices, the similar calculation methodology applied by the National Statistical Office of India (2023).
No API authentication or rate limiting	Enable CORS broadly (allow_origins=["*"]) with no auth or rate limits and the ARIMA endpoint-which re-trains one model per request-is at risk of abuse leading to denial-of-service attacks. In a real world setting, it would be necessary to implement both API key authentication and per-IP rate limits (perhaps using something like slowapi).

7.3 Challenges Faced

Challenge 1: Cross-Layer Schema Consistency.

In our system, five layers of field names span the space between them: CSV column headers title-case with a space (Max Price), the ETL rewrites them to Python naming conventions (max_price), PostgreSQL stores them in lowercase, SQLAlchemy can access them by Python attribute, and Pydantic will serialize them to the field alias corresponding to the Frontend's desired naming conventions. Failure in any of these five areas would lead to silent serialization failure with no obvious error being thrown. In one of the such failures, an error occurred where the date column in the ORM differed from the Pydantic schema alias (record_date). This failure generated an invalid response to the API which only became apparent during integration tests. Our workaround was to introduce a bridging property on the ORM model itself, rather than renaming the column in the database, which would have broken our bulk insert SQL statement. This emphasized how, in multi-layer systems, data contracts have to be explicitly documented and defined up-front, rather than taken to magically propagate.

Challenge 2: ETL & Scheduler integration

Previously DataPipeline was designed to be an executable stand-alone command-line process. To fit into a background task of a FastAPI application this required fixing a Python import-path problem; since dataengineering/ is not inside the kalimatibackend-main/ package it is not found via the normal Python import mechanism. In scheduler.py, the data_engineering path is added to sys.path on import. Although it works, this tightly couples the background process's execution environment to the directory layout of the data engineering layer, an issue which should be addressed by using a proper pipeline orchestration framework in the future.

Challenge 3: Managing In-Memory Data Volume.

To process the first full dataset, we had to maintain all 160,000+ transformed records in memory with a Pandas DataFrame to perform a bulk insert at the end. This was not an issue for our developer workstation with 16GB RAM, but over time as we build more and more daily incremental ETL to our dataset we would have to eventually get close to our

in-memory limit doing a full retransformation from scratch. To avoid this we only selected records that are more recent than the MAX(date) from the database for the incremental flow, so a full reprocessing of the dataset is only necessary on our initial seeded run.

Challenge 4: Managing Team as Project Lead

Besides all the technical difficulties, managing parallel development of frontend, backend analytics, and machine learning was to a significant part determined by a single requirement – to agree on a common API contract very early on, before any team member was actually to consume the API. My task as project lead was to specify the API endpoints, response models and the names of the data attributes as soon as the development starts so both frontend and backend can work in an unblocked fashion. Making sure that the API contract remains stable through multiple changes in data model necessitated not only purely technical engineering communication but also extensive change management.

7.4 Future Improvements

1. Apache Airflow Pipeline Orchestration:

Currently ETL scheduling uses APScheduler, however the approach has no observability of the job status, per stage retries or the ability to run independent of the API server status. By using Apache Airflow each step of the ETL could be an independent task in a Directed Acyclic Graph (DAG), with retry parameters, dependency resolution, and monitoring in a UI dashboard (Chanda, 2024).

2. Redis Caching Layer.

Existing DB-backed analytics caching is still using a single PostgreSQL round trip for each cached analytics request. If we place Redis in front of the existing cache we should see most common product's analytics queries go from ~90ms latency to sub-millisecond and should lower reads on Neon DB.

3. Removing Render Cold Starts.

Cold start latency may be avoided either by simply opting for a Render pro/team plan (keeps the service alive forever), setting up a process with persistent process on a VPS, or sending an automated "keep-warm" ping to the /health endpoint every ten minutes, for example, to avoid idle suspend.

4. Seasonal Decomposition of Imputation.

For future iterations of the imputation algorithm, gaps coinciding with known Nepali public holidays could be identified and a seasonal decomposition algorithm could be utilized - taking expected price path from the corresponding date range in previous years, as opposed to the linear linear path interpolation. This would have given us culturally-informed estimates for the period of time.

5. API Authentication and rate limits.

API keys should be in place for any commercial, and especially government deployment of a solution, with slowapi being utilized for rate limiting the number of requests coming from any single IP address. Slowapi also needs to put a cap on the per-product number of ARIMA forecast calls to keep this part from being exploited by users performing excessive calculations with no immediate revenue incentive.

6. Integrating Multi-Market Data.

Expanding this pipeline to take in data from other significant Nepali wholesale markets (e.g., Bhaktapur, Lalitpur branch) would provide analysis on regional prices and offer a service that could benefit consumers, farmers, traders and agricultural policymakers throughout the Kathmandu Valley.

7.5 Personal Reflection

This project was by far the most technically involved and career shaping project of my undergraduate career, as Project Lead, Data Engineer, and Systems Architect of the Kalimati Market Analytics System. Defining the entire architecture for a system, rather than contributing to a system that was already laid out, required a distinct type of holistic thinking that was different from what it took to build out features.

Perhaps the biggest technical takeaway from this project, however, was a full, practical appreciation of the cascading impact that upstream data quality decisions would have on every layer of downstream system. Upon initial examination of the raw Kalimati CSV files and their seemingly-innocuous data quality issue (e.g., over one hundred different variations for names for what amounts to roughly eighty different products), I treated it as a simple data issue to be scrubbed prior to any other system design. By following a single uncorrected variant (e.g., “Brocauli” instead of “Broccoli”) from initial read into deduplication as an additional product, through to aggregation into its own separate time-series in the database and onto appearing as an unwanted duplicate on the front-end product listing, I began to fully grasp why, in the academic literature, data quality is considered the dominant factor determining the reliability of analytical systems (Chanda, 2024). A “clean” and consistent data layer isn’t an optimization on a functioning system-it is necessary for a functioning one.

My favorite engineering result is the tiered imputation algorithm. It would have been much easier to use the same policy across the board or to simply filter out any row containing nulls. By thinking carefully about the various reasons missing data existed - a single national holiday; a twelve day festival holiday, and a three week data outage - and crafting a solution appropriate to each group i built something that had the nuance one would expect from an end-to-end data product instead of something that was merely “working.” my understanding of the underlying math for each step - linear for slower drift, locf for staying at a point - allowed me to bring principled reasoning to bear on the design instead of simply adopting a convention.

In order to perform the Project Lead duties, I had to grow skill sets outside of software engineering. Setting the API contract before coding started, keeping the schema the same across five translation layers across many iterations, conveying the technical tradeoffs - such as taking on Render’s cold start times to avoid hosting costs - to team members outside of their specialization, managing concurrent workstreams without blocking one another: all technical leaderships lessons that a single engineering task could never teach.

To extend my project further, I plan to deploy the Redis cache layer and the Apache Airflow pipeline described in section 7.4, enabling it from an “academic” production ready application into an” real” production ready tool for public use that can provide price information in a reliable way for the consumers, the market sellers and the government of Nepal.

8. References

- Benchmarking Missing Data Imputation Methods for Time Series Using Real-World Test Cases*. (2025). PubMed Central, National Institutes of Health. <https://www.ncbi.nlm.nih.gov/pmc/>
- Chanda, D. (2024). Automated ETL pipelines for modern data warehousing: Architectures, challenges, and emerging solutions. *The Eastasouth Journal of Information System and Computer Science*, 1(03), 209–212.
- Chippada, S. S., & Agrawal, S. (2025). Modern ETL/ELT pipeline design for ML workflows. *World Journal of Advanced Research and Reviews*, 26(01), 351–358.
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679–688.
- Moritz, S., Sardá, A., Bartz-Beielstein, T., Zaefferer, M., & Stork, J. (2015). Comparison of different methods for univariate time series imputation in R. *arXiv preprint arXiv:1510.03924*. <https://arxiv.org/abs/1510.03924>
- Myers, G. J., Sandler, C., & Badgett, T. (2011). *The art of software testing* (3rd ed.). John Wiley & Sons.
- Nadeem, M. U., et al. (2025). Serverless architecture and its current state of the art: A systematic literature review. *Preprints.org*. <https://www.preprints.org/>
- National Statistical Office of India. (2023). *Handbook on time series imputation methods*. Ministry of Statistics and Programme Implementation, Government of India.
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.
- Udari, W. G. C., & Hemachandra, M. M. (2024). Forecasting vegetable price volatility using ARIMA models: evidence from Sri Lankan wholesale markets. (*Referenced in ForecastService documentation, app/services/forecast_service.py*).